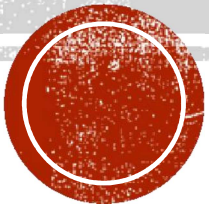
PROJECT THREE: DISTRIBUTED FILE SYSTEM DESIGN



TA: 赵桐(Tong Zhao)

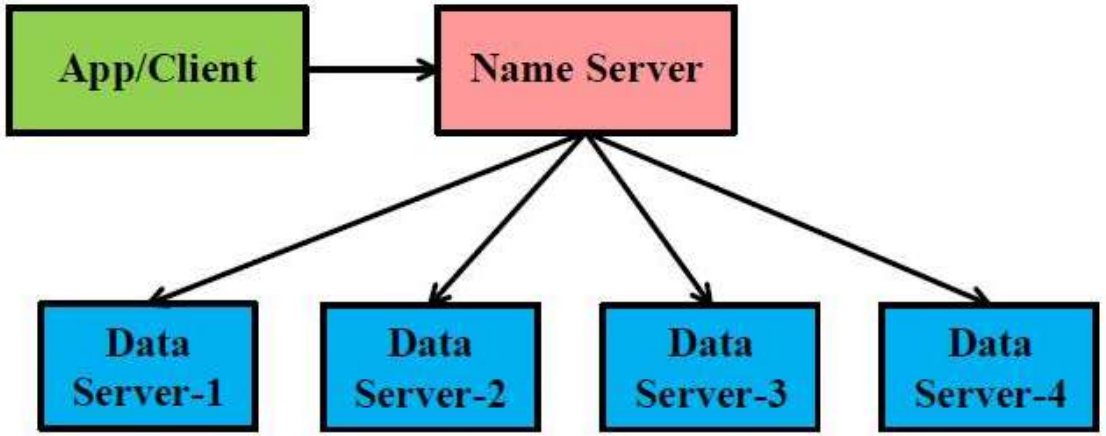
Mail: tongzhao@sjtu.edu.cn

Project Web: http://www.cs.sjtu.edu.cn/~wuct/bdpt/project.html

Dead Line: 2025.11.30

PROJECT THREE

- Design a Mini Distributed File System (Mini-DFS), which contains,
  - A client
  - A name server
  - Four data servers



PROJECT THREE

- Mini-DFS is running through a process. In this process, the name server and data servers are different threads.
- Basic functions of Mini-DFS
  - Read/write a file
    - Upload a file: upload success and return the ID of the file
    - Read the location of a file based on the file ID and the offset
  - File striping
    - Slicing a file into several chunks
    - Each chunk is 2MB
    - Uniform distribution of these chunks among four data servers
  - Replication
    - Each chunk has three replications
    - Replicas are distributed in different data servers



PROJECT THREE

- Name Server
  - List the relationships between file and chunks
  - List the relationships between replicas and data servers
  - Data server management
- Data Server
  - Read/Write a local chunk
  - Write a chunk via a local directory path
- Client
  - Provide read/write interfaces of a file



PROJECT THREE

- Mini-DFS can show
  - Read a file (more than 7MB)
    - Via input the file and directory
  - Write a file (more than 3MB)
    - Each data server should contain appropriate number of chunks
    - Using MD5 checksum for a chunk in different data servers, the results should be the same
    - Check a file in (or not in) Mini-DFS via inputting a given directory
    - By inputting a file and a random offset, output the content



PROJECT THREE

- Bonus points
  - Add directory management
    - Write a file in a given directory
    - Access a file via "directory + file name"
  - Recovery
    - Delete a data server (three data servers survive)
    - Recover the data in the lost data server
    - Redistribute the data and ensure each chunk has three replicas



PLATFORM

- Three Linux PC, Ubuntu system is suggested
- You can also use virtual machines on Windows instead.
- VMWARE and Virtual Box are suggested.
- https://www.virtualbox.org
- https://www.vmware.com





EXAMPLE

- You can use any kind of Programming language like C++,JAVA,PYTHON and so on.
- JAVA may be a better choice and I give a simple example here.



CONNECT

- Use Socket to connect client and server with each other.
- Many open source code can be found on the Internet



REFERENCE ARCHITECTURE

File	Fuction
Client.java	Start Name Server and four data Server thread
NameServer.java	Establish and maintain a directory tree; command parsing and implementation; store md5 checksum of chunk; send read and write chunk commands to data server
DataServer.java	Receive the command from NameServer; read and write chunk
FileTree.java	The implementation class of the directory tree; add folders and files; check if files or folders exist
Flag.java	NameServer and DataServer shared variables; thread synchronization line
Commond.java	NameServer and DataServer shared variables; transfer commands and chunks



DETAILS

- File Storage
  - NameServer divides the file into chunks of 2MB in size and stores them in the three data servers with the least amount of storage.
  - A node is generated at a given location in the directory tree, and a file ID is generated, and the file ID is stored in the node.
  - Each chunk computes the md5 checksum and stores it in the NameServer.
- File Reading
  - NameServer obtains the file ID through the treepath
  - requests the DataServer for the chunk corresponding to the file ID
  - calculates its md5 checksum and compares it with the md5 checksum on the nameserver to obtain the correct chunk and fix the chunks with the md5 checksum error.
  - Get all chunks and write them to the given filepath(local).



DETAILS

- Directory Tree
  - The nodes in the directory tree contain name and fileID. The name can be either a file name or a folder. If the fileID is equal to -1, the node is a folder, otherwise it is a file.



TEST EXAMPLE

- You can use your own testing process and show the results in the report

Step	Results
Upload file to current folder	`miniDFS: /$ put data.tsv d1.tsv` `File upload success. File ID: 0`
Check the current folder	`miniDFS: /$ ls` `d1.tsv  0`
Node Status	Node1, node2, and node3 store 13 consecutive chunks, each of which is 2MB in size, and the content is consistent with data.tsv



TEST EXAMPLE

Step	Results

Create Folder	`miniDFS: /$ mkdir data`
	`miniDFS: /$ ls`
	`data/`
	`d1.tsv  0`

Upload files to the target directory	`miniDFS: /$ put data.tsv data/d2.tsv`
	`File upload success. File ID: 1`
	`miniDFS: /$ cd data`
	`miniDFS: /data/$ ls`
	`d2.tsv  1`

Node Status	Node2, node3, and node4 store 13 consecutive chunks, each of which is 2MB in size, and the content is consistent with data.tsv



TEST EXAMPLE

Step	Results
Check if the file exists	`miniDFS: /$ locate /data/d2.tsv` `/data/d2.tsv exist`

Display file state	`miniDFS: /$ list`
	`File ID Chunks`
	`0 [0_0, 0_1, 0_2, 0_3, 0_4, 0_5, 0_6, 0_7, 0_8, 0_9, 0_10, 0_11, 0_12, ]`
	`1 [1_0, 1_1, 1_2, 1_3, 1_4, 1_5, 1_6, 1_7, 1_8, 1_9, 1_10, 1_11, 1_12, ]`
	`File ID Data Servers`
	`0 [0, 1, 2]`
	`1 [3, 1, 2]`



TEST EXAMPLE

Step	Results
Read file	
Read last two chunk	
Data parity	…
Data recovery	…

